Set up your computer for VisionIQ

Team onboarding | Windows 11 / PowerShell | One-time setup
Repository: https://github.com/sandeshshinde284-bit/visionIQ
Document version: 29 October 2026 | **Repository reference:** main at 73307f1

Use this guide when preparing a developer computer for VisionIQ. It covers the local development stack: **ASP.NET Core 8 API (port 5100), Python 3.12 FastAPI service (port 8000), and PostgreSQL 16 with pgvector**. After the first successful setup, use the separate **Run VisionIQ Locally** guide for everyday commands.

> **Scope:** A local development environment does **not** require Google Cloud sign-in, Gemini/Claude credentials, Azure DevOps, Jira or Confluence connectivity. These integrations are later work. Follow your organization's software and credentials policies.

1. Clone the current repository

Where: Open PowerShell in any approved development folder. Do not use a hard-coded personal drive path.

```
git --version
git clone https://github.com/sandeshshinde284-bit/visionIQ.git
Set-Location .\visionIQ
git branch --show-current
Test-Path .\scripts\check-env.ps1
```

Expected: The branch is `main`; the script check prints `True`. If already cloned, open a terminal in the repository root and run `git pull origin main` after checking for uncommitted work. Clone access must be arranged through the approved GitHub process; never paste personal access tokens into the guide.

2. Check prerequisites and install missing tools

Where: Repository root. The supplied checker verifies installed tools and recommends fixes. Plain mode changes nothing; `-Install` offers approved installations and creates `.env` only when it is missing. It never sets a database password automatically.

```
powershell -NoProfile -ExecutionPolicy Bypass -File .\scripts\check-env.ps1
# Only if tools are missing:
powershell -NoProfile -ExecutionPolicy Bypass -File .\scripts\check-env.ps1 -Install
```

Required for the application: **Git, .NET 8 SDK, Python 3.12, Docker Desktop with a running engine and Compose**, and an editor such as Visual Studio or VS Code. The repository pins Python using `.python-version` and .NET using `global.json`. Start Docker Desktop; if software was installed, open a new PowerShell window and repeat the check.

Expected: Required tools show `[OK]`. The checker also inspects Google Cloud CLI; a missing cloud login/project is **not** a blocker for basic local app startup. Follow organization IT processes if installation needs admin/WSL2 approval.

3. Create your private environment file

Where: Repository root. The template is committed; each developer's actual `.env` is private and excluded from Git.

```
if (-not (Test-Path .env)) { Copy-Item .env.example .env }
notepad .env
```

Set `DB_LOCAL_PASSWORD` to a strong, nonempty **local-only** password. Keep `DB_TARGET=local`. With the template defaults, the local database uses `DB_LOCAL_NAME=visioniq`, `DB_LOCAL_USER=visioniq` and `DB_LOCAL_PORT=5432`. Leave cloud and external-provider keys blank until an approved task needs them.

Important: If an existing PostgreSQL Docker volume was initialized with a different password, merely editing `.env` does **not** change the database's stored password. Ask before modifying an existing developer database; do not erase its volume to repair a password mismatch.

4. Select the local database and start the complete stack

Where: Repository root. `use-db.ps1 local` updates the effective `DB_*` keys in `.env`; it does not start Docker. The current Compose file starts **all three services** with a single command.

```
powershell -NoProfile -ExecutionPolicy Bypass -File .\scripts\use-db.ps1 local
docker compose config --quiet
docker compose up --build -d
docker compose ps -a
```

Expected: `visioniq-postgres`, `visioniq-api` and `visioniq-intelligence` exist and become **healthy**. The first build downloads dependencies and may take longer than subsequent starts. If a container exits or becomes unhealthy, use `docker compose logs --tail=100 api intelligence postgres` and follow the everyday running guide.

5. Verify the two local APIs

Where: Repository root or any PowerShell window on the same machine.

```
(Invoke-WebRequest http://localhost:5100/health).StatusCode
(Invoke-WebRequest http://localhost:8000/health).StatusCode
```

Expected: `200` for both. These are **liveness** checks: they verify that each application responds, not that its database, AI providers or external integrations are working.

6. Verify local PostgreSQL and pgvector

Where: Repository root. These commands use the default local user/database from `.env.example`; substitute your chosen values if different.

```
docker compose exec postgres psql -U visioniq -d visioniq -c "SELECT 1;"
docker compose exec postgres psql -U visioniq -d visioniq -c "\dx"
docker compose exec postgres psql -U visioniq -d visioniq -c "\dn"
```

Expected: `SELECT 1` succeeds; `\dx` lists the `vector` extension; `\dn` lists the initialized schemas. The initialization SQL runs **only when a fresh database volume is created**. Broader schema ownership and migrations are tracked separately from local-environment setup.

7. Optional: Google Cloud access for an authorized cloud task

Skip this section for local FND-02 testing. If a later task requires Google Cloud, sign in using an organization-approved account and select only the authorized VisionIQ project. A dedicated CLI configuration avoids changing another project's active settings.

```
gcloud auth list
gcloud config configurations create visioniq-dev
# If already created, use instead:
# gcloud config configurations activate visioniq-dev
gcloud auth login
gcloud projects list --format="table(projectId,name)"
gcloud config set project APPROVED_VISIONIQ_PROJECT_ID
gcloud projects describe APPROVED_VISIONIQ_PROJECT_ID
gcloud auth application-default login
```

Replace the project placeholder with the **approved** ID; do not execute a placeholder literally. Google Cloud CLI authentication and Application Default Credentials (ADC) are separate. These commands do not deploy anything.

Never commit tokens, service-account JSON files or private `.env` values.

8. Finish onboarding

Run `docker compose ps -a` and confirm the three services are healthy, then verify both HTTP 200 responses. For test/lint commands, rebuilds, logs, shutdown and working without Docker application containers, see **Run VisionIQ Locally**.

FND-02 acceptance: A fresh checkout must start all three services with one Compose command; both health endpoints return 200; local .NET build/xUnit, Python pytest/Ruff and .NET formatting/analyzer checks pass. CI/PR validation is **outside the currently agreed FND-02 Trello card scope** and should be tracked separately.